DOCUMENTAÇÃO DE TESTES

Volunjá API — Issue #20: Consultar e Atualizar Perfil do Voluntário Autenticado

Arquivo de referência	`tests/auth/test_voluntarios_me.py`
Frameworks e bibliotecas	pytest, unittest.mock, FastAPI TestClient, PyJWT
Escopo documentado	Rotas HTTP `/api/v1/voluntarios/me` (GET e PUT), autenticação JWT e conversão de exceções de perfil
Quantidade de cenários	6 casos de teste automatizados (CT-01 a CT-06)

1. Resumo da Issue

Implementação e validação dos endpoints `GET /api/v1/voluntarios/me` e `PUT /api/v1/voluntarios/me`, permitindo que o voluntário autenticado consulte e atualize seu próprio perfil. A identidade do usuário é derivada exclusivamente da *claim* `sub` presente no token JWT fornecido no cabeçalho `Authorization`, impedindo qualquer injeção de parâmetros externos.

2. Detalhamento dos Endpoints

Método	Rota	Descrição	Sucesso
`GET`	`/api/v1/voluntarios/me`	Retorna dados pessoais, causas e disponibilidades do voluntário autenticado.	200 OK
`PUT`	`/api/v1/voluntarios/me`	Atualiza parcialmente os campos mutáveis do perfil do usuário autenticado.	200 OK

Respostas de erro padrão: `401 Unauthorized` (token ausente/expirado/inválido), `403 Forbidden` (cabeçalho de autorização ausente) e `404 Not Found` (voluntário inexistente ou inativo).

3. Estrutura em Camadas

- **schemas:** `AtualizaPerfilDeUsuariosRequest` (campos opcionais para suporte a atualização parcial via `model_dump(exclude_unset=True)`) e `PerfilVoluntarioResponse`.
- **repository:** `buscar_perfil_por_id` e `atualizar_perfil` (consultas PyMySQL sobre a tabela `usuarios` filtrando exclusivamente contas ativas).
- **service:** `obter_perfil_por_id` e `atualizar_perfil`, com tratamento da exceção de negócio `VoluntarioNaoEncontradoException`.
- **router:** Endpoints sob o prefixo `/me`, injeção de dependência do JWT e tradução de exceções do serviço para status HTTP.

4. Regras de Negócio Testadas

- **Identidade via JWT:** A chave identificadora do voluntário é estritamente obtida da *claim* `sub`.
- **Campos Mutáveis:** Apenas `nome_completo`, `telefone`, `cidade`, `estado`, `biografia`, `causas` e `disponibilidades` podem ser alterados.
- **Imutabilidade e Segurança:** Tentativas de enviar `id`, `email`, `identificacao`, `perfil` ou `password` no payload são descartadas pelo schema. As respostas nunca expõem `password_hash` ou `password`.
- **Estruturas Complexas:** As propriedades `causas` e `disponibilidades` são persistidas como JSON e devolvidas como listas de strings.
- **Inatividade:** Um voluntário com status inativo é tratado como não encontrado (retornando status 404).

5. Tecnologias e Recursos de Teste

pytest	Framework de execução, fixtures e validações comportamentais.
unittest.mock.MagicMock	Isolamento da camada de serviço por meio de `app.dependency_overrides`.
FastAPI TestClient	Simulação de requisições HTTP aos endpoints da aplicação sem servidor ativo.
PyJWT (jwt)	Geração de tokens assinalados (válidos ou expirados) para cenários de autenticação.

6. Funções Auxiliares dos Testes

Função	Finalidade / Comportamento
`gerar_token_teste(sub, expirado=False)`	Gera um JWT assinado válido com as configurações do serviço (`SECRET_KEY` e `ALGORITHM`). Se `expirado=True`, define a *claim* `exp` no passado.
`mock_perfil_padrao()`	Instancia e retorna um objeto `PerfilVoluntarioResponse` preenchido com dados padrão para simular o retorno do serviço nas rotas.

7. Casos de Teste Detalhados

7.1. Endpoints GET /api/v1/voluntarios/me

ID	Função de Teste	Cenário e Verificação	Esperado
CT-01	`test_get_me_sucesso_retorna_perfil_do_proprio_usuario`	Envia requisição com token JWT válido. O service mockado retorna o perfil do usuário. Confere se os dados retornados coincidem, se `password` e `password_hash` foram omitidos e se o service foi chamado exatamente uma vez com o ID do `sub`.	HTTP 200 OK

ID	Função de Teste	Cenário e Verificação	Esperado
CT-02	`test_get_me_sem_cabecalho_authorization_retorna_403`	Envia requisição GET para `/me` sem incluir o cabeçalho `Authorization`. Verifica se a camada de segurança bloqueia o acesso não autenticado.	HTTP 403 Forbidden
CT-03	`test_get_me_com_token_expirado_retorna_401`	Envia requisição com um token JWT cuja data de expiração (`exp`) está no passado. Confere se o retorno rejeita o token com mensagem informando "Token expirado".	HTTP 401 Unauthorized
CT-04	`test_get_me_usuario_nao_encontrado_no_banco_retorna_404`	Token JWT válido é enviado, mas o serviço lança `VoluntarioNaoEncontradoException`. Confere se a rota converte o erro para 404 com a mensagem "Voluntário não encontrado".	HTTP 404 Not Found

7.2. Endpoints PUT /api/v1/voluntarios/me

ID	Função de Teste	Cenário e Verificação	Esperado
CT-05	`test_put_me_sucesso_atualiza_dados_pessoais_causas_e_disponibilidades`	Envia payload com novos valores para nome, cidade, causas e disponibilidades. O serviço mockado processa a alteração. Verifica se os dados atualizados retornam no JSON de resposta e se a função `atualizar_perfil` foi invocada.	HTTP 200 OK
CT-06	`test_put_me_ignora_e_preserva_campos_imutaveis_e_perfil`	Envia corpo contendo campos imutáveis (`id` , `email` , `perfil` , `identificacao` , `password`). Verifica se o schema ignora esses campos, mantendo o perfil e identidade originais inalterados.	HTTP 200 OK

8. Mapeamento de Cobertura dos Critérios de Aceitação

Critério de Aceite (Issue #20 / CN02-RF06)	Casos de Teste Associados
Extrair e utilizar exclusivamente a identidade vinda do claim `sub` do JWT	**CT-01**
Retornar apenas os dados permitidos do perfil, omitindo dados sensíveis de credencial	**CT-01**
Rejeitar requisições sem autenticação ou fornecidas com token expirado/inválido	**CT-02**, **CT-03**
Tratar adequadamente o cenário de voluntário inexistente ou com conta inativa	**CT-04**

Critério de Aceite (Issue #20 / CN02-RF06)	Casos de Teste Associados
Validar e persistir corretamente dados pessoais, causas e disponibilidades mutáveis	**CT-05**
Garantir a preservação dos campos imutáveis do registro e do perfil de acesso	**CT-06**

9. Instruções de Execução dos Testes

Para executar a suíte focada especificamente nos cenários de consulta e edição de perfil (Issue #20), navegue até a pasta do projeto (`src/backend`) e execute:

```
pytest -v -k "get_me or put_me"
```

Para executar toda a suíte de testes de voluntários com emissão do relatório de cobertura de código:

```
PYTHONPATH=. pytest tests/auth/ -v --cov=app/voluntarios --cov-report=term-missing
```

10. Exemplo da Implementação do Arquivo de Testes

Abaixo apresenta-se a estrutura base do arquivo `tests/auth/test_voluntarios_me.py` contendo as fixtures e os cenários automatizados:

```
import time import jwt import pytest from unittest.mock import MagicMock from fastapi.testcl
ient import TestClient from app.voluntarios.main import app from app.voluntarios.router impo
rt get_voluntario_service from app.voluntarios.schemas import PerfilVoluntarioResponse from
app.voluntarios.service import SECRET_KEY, ALGORITHM, VoluntarioNaoEncontradoException def g
erar_token_teste(sub="usr_001", expirado=False): exp = time.time() - 3600 if expirado else t
ime.time() + 3600 payload = {"sub": sub, "email": "maria@teste.com", "perfil": "Voluntario",
"exp": exp} return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM) def mock_perfil_padr
ao(): return PerfilVoluntarioResponse( id="usr_001", nome_completo="Maria de Souza", email
="maria@teste.com", identificacao="99988877766", telefone="11999998888", cidade="São Paulo",
estado="SP", biografia="Voluntária ativa", causas=["Educação"], disponibilidades=["Finais de
semana"] ) def test_get_me_sucesso_retorna_perfil_do_proprio_usuario(): service_mock = Magic
Mock() service_mock.obter_perfil_por_id.return_value = mock_perfil_padrao() app.dependency_o
verrides[get_voluntario_service] = lambda: service_mock client = TestClient(app) headers =
{"Authorization": f"Bearer {gerar_token_teste('usr_001')}"} res = client.get("/api/v1/volunt
arios/me", headers=headers) assert res.status_code == 200 data = res.json() assert data["i
d"] == "usr_001" assert "password" not in data and "password_hash" not in data service_mock.
obter_perfil_por_id.assert_called_once_with("usr_001") def test_get_me_sem_cabecalho_authori
zation_retorna_403(): client = TestClient(app) assert client.get("/api/v1/voluntarios/me").s
tatus_code == 403 def test_get_me_com_token_expirado_retorna_401(): client = TestClient(app)
headers = {"Authorization": f"Bearer {gerar_token_teste(expirado=True)}"} res = client.get
("/api/v1/voluntarios/me", headers=headers) assert res.status_code == 401 assert "Token expi
rado" in res.json()["detail"] def test_get_me_usuario_nao_encontrado_no_banco_retorna_404():
service_mock = MagicMock() service_mock.obter_perfil_por_id.side_effect = VoluntarioNaoEncon
tradoException("Voluntário não encontrado") app.dependency_overrides[get_voluntario_service]
= lambda: service_mock client = TestClient(app) headers = {"Authorization": f"Bearer {gerar_
token_teste('usr_999')}"} res = client.get("/api/v1/voluntarios/me", headers=headers) assert
res.status_code == 404 assert "Voluntário não encontrado" in res.json()["detail"]
```

11. Modelo para Descrição de PR

```
feat: implementa consulta e atualização do perfil do voluntário (Issue #20) - Adiciona GET /
me e PUT /me com autenticação JWT. - Adiciona métodos de busca e atualização parcial no repo
sitory. - Adiciona obter_perfil_por_id e atualizar_perfil no service. - Adiciona testes auto
matizados (CT-01 a CT-06) com service mockado. Closes #20
```